

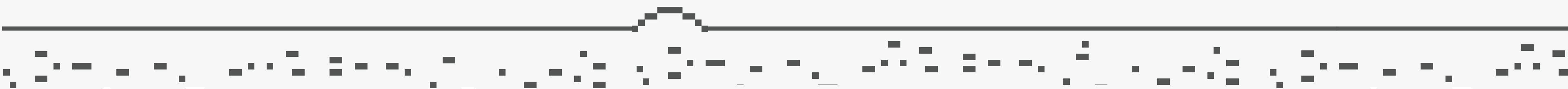
SRMP MONITOR

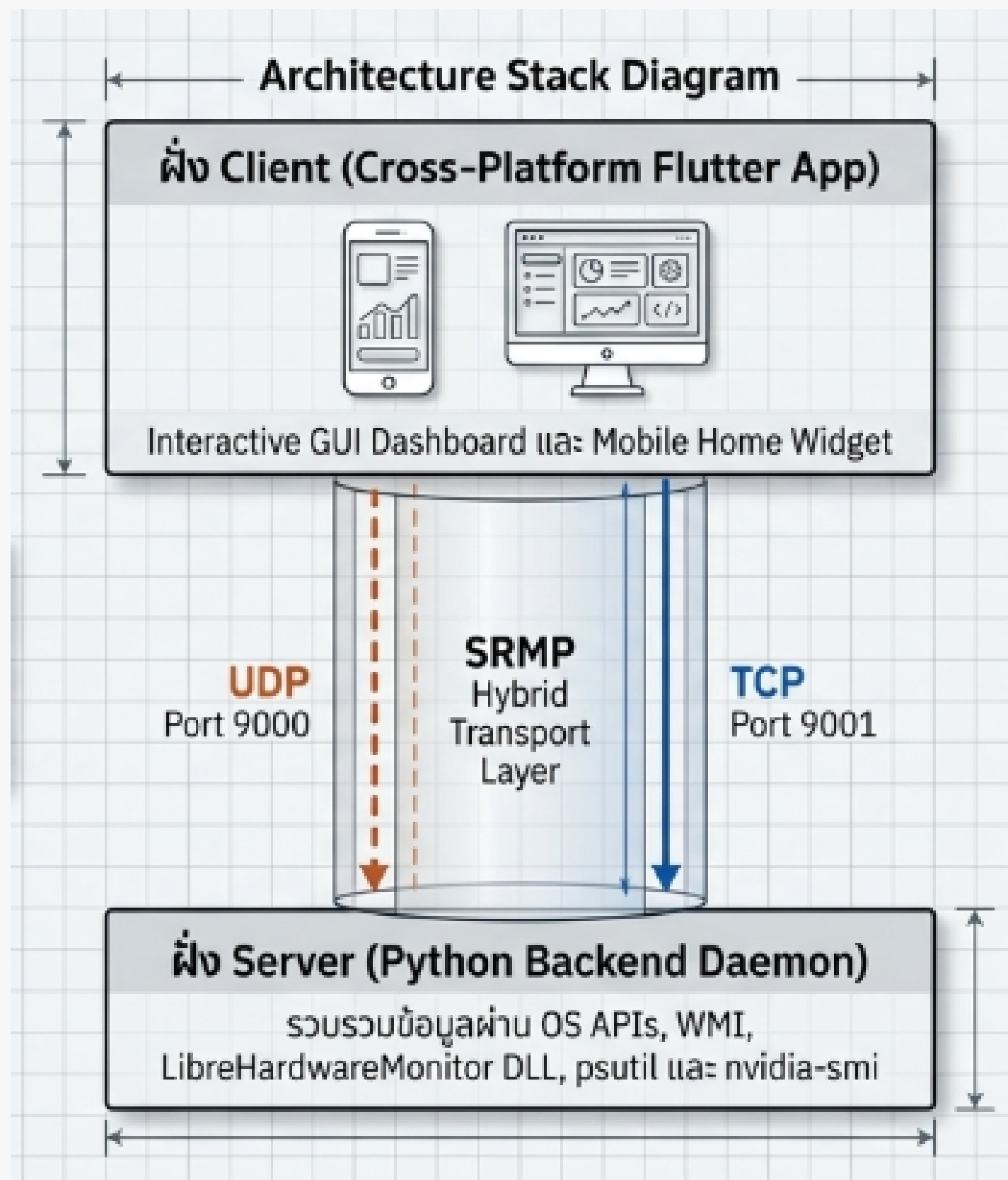
(SYSTEM RESOURCE MONITORING PROTOCOL)



วัตถุประสงค์การทำงานของโปรแกรม

- การติดตามสถานะทรัพยากรฮาร์ดแวร์แบบ Real-Time
ดึงข้อมูลสถานะการทำงานระดับลึก ได้แก่ อัตราการใช้
งาน CPU (%), RAM (%), Disk Storage (%)
- การควบคุมระบบและฮาร์ดแวร์จากระยะไกล
ปรับระดับเสียง ความสว่างหน้าจอได้ และควบคุมการปิด
เปิดหน้าจอจากระยะไกล
- การบริหารจัดการ Process
ตรวจสอบรายการโปรเซสที่ใช้ทรัพยากรสูงสุดเรียงลำดับ
ตาม CPU หรือ RAM และสามารถสั่งยุติการ
- ระบบแจ้งเตือนเหตุฉุกเฉิน
แจ้งเตือนทันทีเมื่อตรวจพบว่าทรัพยากรของ เครื่องทำ
งานหนักเกินเกณฑ์





คุณลักษณะของ Application

Dual-Layer Client-Server Model

- ฝั่ง Server (Python Backend): ทำหน้าที่คอยเก็บข้อมูลจากระดับ Operating System และคอยรับคำสั่งควบคุมระบบ
- ฝั่ง Client (Cross-Platform Flutter App): ทำหน้าที่เป็นหน้าจอควบคุม รองรับทั้งสมาร์ทโฟนและคอมพิวเตอร์

การเลือกใช้ Service Model ของ Transport Layer

- ใช้ สถาปัตยกรรมแบบไฮบริด (Hybrid Transport Layer Architecture) โดยแบ่ง การทำงาน ออกเป็น 2 ท่อ

UDP

- ใช้สำหรับ Metric Telemetry Broadcast และ Wake-on-LAN
- เพราะรวดเร็ว ยอมรับการสูญหายของ ข้อมูลได้ และสามารถส่งแบบ Broadcast

TCP

- ใช้สำหรับ Command & Control Channel และ Alert Notifications
- เพราะการันตีว่าข้อมูลถึงปลายทาง รักษาลำดับความถูกต้องของคำสั่ง และตรวจสอบสถานะ Online/Offline ของกันและกันได้

UDP AND TCP TABLE

มิติการเปรียบเทียบ	UDP Channel (Port 9000)	TCP Channel (Port 9001)
ฟังก์ชันการทำงาน	สตรีมข้อมูลสถิติ (Metrics) & Wake-on-LAN	คำสั่งควบคุม (Commands), ผลตอบกลับ (Response), แจ้งเตือน (Alerts)
ลักษณะการเชื่อมต่อ	Connectionless (ส่งทันทีโดยไม่ต้องเชื่อมต่อ)	Connection-Oriented (Three-Way Handshake)
การรับประกันความน่าเชื่อถือ	Unreliable (ไม่การันตี, ไม่มีการส่งซ้ำ)	Reliable (รับประกันข้อมูลถึงครบ 100%, มี Retransmission)
ทิศทางและรูปแบบการส่ง	One-Way Broadcast (1-to-Many)	Two-Way Interactive Unicast (Client ↔ Server)
Overhead & Header Size	ต่ำมาก (Header 8 Bytes)	ปานกลาง (Header 20 Bytes + ควบคุมสถานะท่อเชื่อมต่อ)

การออกแบบ Protocol

- **ชื่อโปรโตคอล:** SRMP (System Resource Monitoring Protocol) v1.0
- **รูปแบบข้อความ (Message Encoding):** Plain-text ASCII / UTF-8 String
- **UDP Metric Broadcast:** Server ทำการ Broadcast ข้อมูลไปยัง Port 9000 ทุกๆ 1 วินาที โดยส่งเป็นข้อความแบบ Plain-Text

รูปแบบข้อความ: METRIC cpu=<CPU%> ram=<RAM%> disk=<DISK%> temp_cpu=<TEMP_C> temp_gpu=<TEMP_C> net_up=<MB/s> net_down=<MB/s> net_online=<0|1> net_ping=<MS> uptime=<SECS>

ตัวอย่างข้อความที่ส่งจริง:

METRIC cpu=24.5% ram=68.2% disk=45.0% temp_cpu=58.5C temp_gpu=46.0C net_up=0.12MB/s
net_down=1.45MB/s net_online=1 net_ping=18ms uptime=43200s

ความหมายของข้อมูล: CPU ทำงาน 24.5%, RAM 68.2%, Disk 45.0%, ความร้อน CPU 58.5°C, ความร้อน GPU 46.0°C, ความเร็วเน็ต Up 0.12 MB/s, Down 1.45 MB/s, สถานะเน็ตออนไลน์ ปิงเฉลี่ย 18ms, เครื่องเปิดใช้งานมาแล้ว 43,200 วินาที (12 ชั่วโมง)

โครงสร้างคำสั่งและข้อความตอบกลับ TCP

- **Request Message Syntax:** [arg1=value1] [arg2=value2]...
- **Response Message Syntax:** <STATUS_CODE> <STATUS_PHRASE> - <RESPONSE_BODY>

หมวดหมู่	Request Message (Client → Server)	Response Message (Server → Client)
Process List	<code>GET_TOP_PROCS</code> <code>limit=8</code> <code>sortby=CPU</code> (<i>sortby รองรับ CPU หรือ RAM</i>)	<code>200 OK - procs=</code> <code>[{pid:1240,name:chrome.exe,cpu:15.2%,ram:8.4%,ram_mb:1344.2MB},...]</code>
Kill Process	<code>KILL_PROC</code> <code>pid=1240</code>	<code>200 OK - PROC_KILLED</code> หรือ <code>404 NOT_FOUND - PROCESS_NOT_EXIST</code>
Get Setting	<code>GET_SETTING</code> <code>name=volume</code> <code>GET_SETTING</code> <code>name=brightness</code>	<code>200 OK - SETTING_VALUE name=volume value=65</code>

หมวดหมู่	Request Message (Client → Server)	Response Message (Server → Client)
Set Volume	SET_VOL level=75	200 OK - VOLUME_SET_75
Set Brightness	SET_BRIGHTNESS level=80	200 OK - BRIGHTNESS_SET_80
System Power	SYS_POWER action=LOCK SYS_POWER action=RESTART SYS_POWER action=SHUTDOWN	200 OK - SYSTEM_LOCKED 200 OK - SYSTEM_RESTARTING 200 OK - SYSTEM_SHUTDOWN
Server- Push Alert	(ส่งแจ้งเตือนมาที่ Server เมื่อเกิดเหตุ ฉุกเฉิน)	202 ALERT - CPU Temperature exceeded 85C threshold

3.5 แผนผังลำดับขั้นตอนการสื่อสาร (Message Exchange Flow Diagram)

```
[ Client (Flutter App) ]                                [ Server (Python Daemon) ]
|
| ----- 1. UDP Broadcast Listener (Port 9000) ----- | (Listening)
| <..... UDP Packet: METRIC cpu=24% ram=68% ..... | (Every 1 sec)
|
| ===== 2. TCP Handshake & Connect (Port 9001) ===== |
| -----> | (Connection Established)
|
| ----- GET_TOP_PROCS limit=8 sortby=CPU -----> |
| <----- 200 OK - procs=[{pid:1240,name:...}] ----- |
|
| ----- SET_VOL level=75 -----> |
| <----- 200 OK - VOLUME_SET_75 ----- |
|
| ----- KILL_PROC pid=1240 -----> |
| <----- 200 OK - PROC_KILLED ----- |
|
| <----- 202 ALERT - CPU Temp High (88°C) ----- | (Push Event)
```